SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO3 ASSESSMENT TOOL 1 – Git & Docker Technical Assignment

Course Code:	CSA10 / CSA1011	**Course Title:**	Software Engineering
CO Assessed:	CO3	**Assessment:**	Assessment Tool 1 – Git & Docker Technical Assignment
Weightage:	20%	**Total Marks:**	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	HARIHARAN.R	**Reg. No.:**	192524003
Date:	28/10/2026	**Duration:**	60 minutes

Code of Conduct

I _____HARIHARAN.R (192524003)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: HARIHARAN.R_____

Annexure A – Git & Docker Technical Assignment

Instructions to Students:

Each student has been assigned an individual problem statement. Based on the assigned problem statement, prepare a **Git & Docker Technical Assignment** by applying version control and containerization concepts. Your report should include appropriate diagrams, screenshots, commands, and explanations wherever applicable.

Assignment Questions

1. **Problem Analysis**
   - Explain the assigned problem statement and identify the project objectives, functional requirements, and expected outcomes.
2. **Project Structure Design**
   - Design and explain the folder structure of your project repository.
   - Justify the purpose of each major folder and file included in the repository.
3. **Git Repository Initialization**
   - Create a Git repository for the project.
   - Explain the steps involved in repository initialization and describe the purpose of the essential Git files.
4. **Git Branching Strategy**
   - Design a suitable branching strategy (e.g., Main, Development, Feature, Release, Hotfix).
   - Justify why the selected branching model is appropriate for the assigned project.
5. **Version Control Workflow**
   - Demonstrate the Git workflow by performing meaningful commits, branch creation, merging, conflict resolution (if applicable), and repository synchronization.

    - Explain the purpose of each Git operation performed.
6. **Commit History Analysis**
    - Present the commit history of the project.
    - Explain how commit messages follow good version control practices and contribute to project maintenance.
7. **Docker Environment Design**
    - Explain why Docker is suitable for the assigned project.
    - Identify the application components that require containerization.
8. **Dockerfile Development**
    - Design and explain the Dockerfile required to containerize the application.
    - Describe the purpose of each instruction used in the Dockerfile.
9. **Docker Compose Design**
    - Develop a Docker Compose configuration for the project.
    - Explain the services, networking, volumes, environment variables, and dependencies used.
10. **Container Deployment and Testing**
    - Demonstrate the execution of the application using Docker containers.
    - Explain the testing procedure and provide evidence that the application runs successfully.
11. **Benefits of Git and Docker**
    - Discuss how Git and Docker improve collaboration, version management, portability, deployment, scalability, and maintainability for your assigned project.
12. **Challenges and Improvements**
    - Identify the challenges encountered during implementation.
    - Suggest possible improvements and future enhancements for the Git workflow and Docker deployment.

Expected Deliverables

- Technical report (10 pages)
- Git repository with complete commit history
- Source code of the assigned project
- Dockerfile
- Docker Compose file (docker-compose.yml)
- Screenshots of Git operations and Docker execution
- Architecture and workflow diagrams

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Project Design(20%)	Clearly explains the problem statement, objectives, requirements, expected outcomes, and well-organized project structure with proper justification.	Good analysis and project structure with minor omissions.	Basic analysis and repository structure with limited justification.	Incomplete or unclear analysis and project organization.

Git Repository & Branching Strategy(20%)	Repository initialized correctly with appropriate branching strategy, clear workflow, and proper repository organization.	Repository and branching strategy are mostly correct.	Basic Git setup with limited branching implementation.	Incorrect or incomplete Git repository and branching strategy.
Version Control Workflow & Commit History(10%)	Demonstrates meaningful commits, branching, merging, synchronization, conflict resolution, and professional commit messages with clear history.	Most Git operations demonstrated correctly with good commit history.	Limited Git workflow and commit practices.	Poor workflow and inadequate commit history.
Docker Implementation(20%)	Well-designed Docker environment, Dockerfile, and Docker Compose configuration with clear explanation of services, networking, volumes, and dependencies.	Functional Docker implementation with minor issues.	Basic Docker implementation with limited explanation.	Incomplete or incorrect Docker implementation.
Deployment, Testing & Results(15%)	Application successfully deployed and tested with appropriate screenshots and evidence of successful execution.	Deployment completed with minor issues.	Partial deployment and testing demonstrated.	Deployment unsuccessful or insufficient evidence.
Documentation, Challenges & Future Enhancements(15%)	Professional report (10–15 pages) with diagrams, screenshots, references, discussion of benefits, challenges, and future improvements.	Good documentation with minor omissions.	Basic documentation with limited discussion.	Poor documentation and insufficient analysis.

Criteria	**Mark**
Problem Analysis & Project Design (20%)	/5
Git Repository & Branching Strategy(20%)	/4
Version Control Workflow & Commit History(10%)	/4
Docker Implementation(20%)	/4
Deployment, Testing & Results(15%)	/4
Documentation, Challenges & Future Enhancements(15%)	/4
TOTAL	/25

Food Delivery Application: Design an architecture for a food delivery system involving customers, restaurants, delivery partners, order management, and payment processing.

ANSWER::

1. AIM

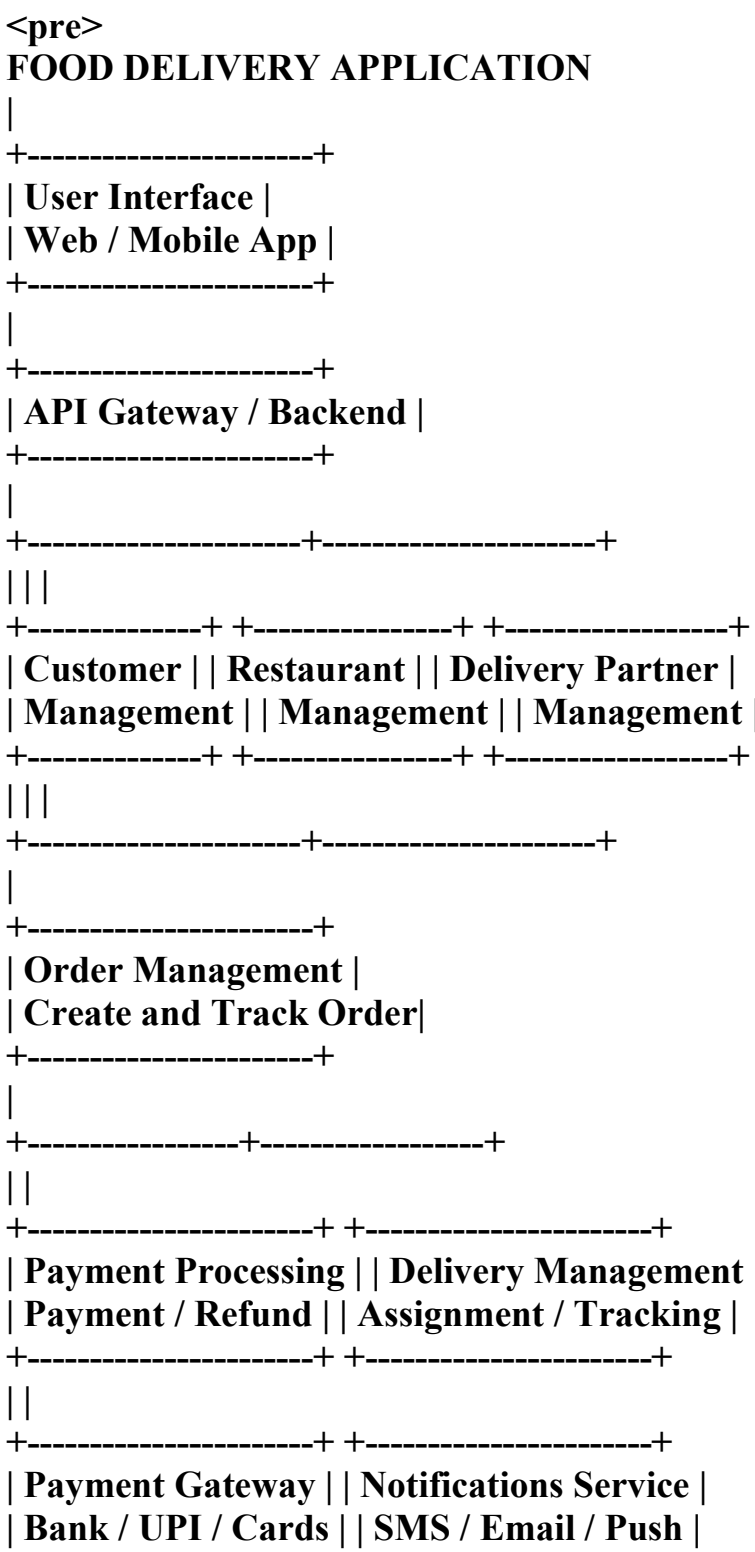
To design a scalable and reliable architecture for a Food Delivery Application that connects customers, restaurants, delivery partners, order management, and payment processing using suitable software components. The application is designed to manage food ordering, payment transactions, restaurant preparation, delivery tracking, and order status updates efficiently. Git is used for version control, and Docker is used for application containerization and deployment.

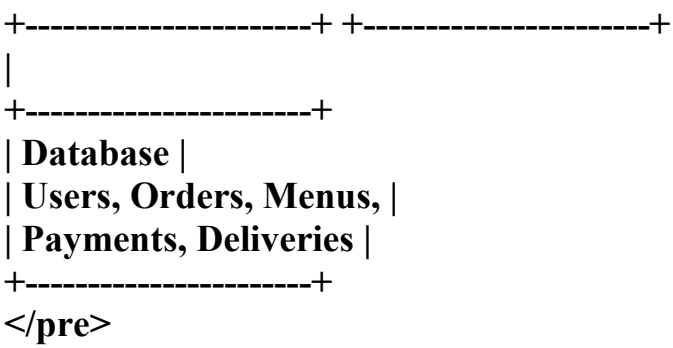
2. INTRODUCTION

A Food Delivery Application is an online platform that allows customers to browse restaurants, view menus, place food orders, make online payments, and track deliveries. Restaurants can manage menus and accept orders, while delivery partners can receive delivery requests and update delivery status.
The system consists of multiple interconnected modules that communicate through APIs. The architecture ensures secure payment processing, efficient order management, real-time status updates, and reliable communication between all users.

3. SYSTEM ARCHITECTURE

Architecture Diagram:





</pre>

4. MAJOR COMPONENTS OF THE SYSTEM

1. Customer Management Module
   - Allows customers to register and log in securely.
   - Displays nearby restaurants and available food items.
   - Enables customers to add food items to the cart and place orders.
   - Provides order history, delivery tracking, and cancellation options.
2. Restaurant Management Module
   - Allows restaurants to manage their profiles and food menus.
   - Displays food prices, descriptions, and availability.
   - Enables restaurants to accept or reject orders.
   - Updates food preparation status and estimated preparation time.
3. Delivery Partner Management Module
   - Stores delivery partner details and availability.
   - Assigns orders to suitable delivery partners.
   - Provides pickup and delivery information.
   - Allows delivery partners to update order status, such as picked up and delivered.
4. Order Management Module
   - Receives and validates customer orders.
   - Calculates the total price, delivery charges, and applicable taxes.
   - Sends order details to the restaurant.
   - Tracks the order from placement to successful delivery.
   - Handles order cancellation and refund requests according to system rules.
5. Payment Processing Module
   - Supports online payment methods such as UPI, debit cards, credit cards, and net banking.
   - Connects securely with an external payment gateway.
   - Verifies payment results before confirming prepaid orders.
   - Maintains transaction records and supports refunds.
   - Avoids duplicate payment processing through transaction verification and idempotency controls.
6. Notification and Tracking Module

- **Sends order confirmation messages to customers.**
- **Notifies restaurants about new orders.**
- **Informs delivery partners about assigned deliveries.**
- **Sends status updates through push notifications, SMS, or email.**
- **Displays delivery progress and estimated arrival time.**

7. Database Management Module

- **Stores customer, restaurant, menu, order, payment, and delivery information.**
- **Maintains relationships between orders, customers, and restaurants.**
- **Uses appropriate access controls and backup mechanisms.**
- **Supports efficient retrieval and updating of application data.**

5. WORKING PROCESS OF THE FOOD DELIVERY SYSTEM

Step 1: Customer Login
The customer registers or logs in to the application and provides the delivery address.
Step 2: Restaurant Selection
The system displays available restaurants, menus, food prices, and estimated delivery times.
Step 3: Order Placement
The customer selects food items, adds them to the cart, and confirms the order.
Step 4: Payment Processing
The system calculates the bill and sends the payment request to a secure payment gateway. The order is confirmed for prepaid transactions after successful payment verification.
Step 5: Restaurant Confirmation
The restaurant receives the order, accepts it, and begins preparing the food.
Step 6: Delivery Partner Assignment
The system assigns an available delivery partner based on factors such as location and availability.
Step 7: Food Pickup and Delivery
The delivery partner collects the food and delivers it to the customer's address. The application updates the delivery status.
Step 8: Order Completion
After delivery, the system marks the order as completed and allows the customer to provide feedback and ratings.

6. TECHNOLOGIES USED

Technology	Purpose
HTML, CSS, JavaScript	User interface development
Python Flask / Node.js	Backend application development
REST APIs	Communication between modules
MySQL / PostgreSQL	Database management
Git and GitHub	Version control and code collaboration
Docker	Application containerization
Payment Gateway API	Secure payment processing
WebSocket / Push Notifications	Real-time updates

7. GIT IMPLEMENTATION

Git is used to track source code changes, maintain different versions of the application, and support collaboration among developers.

Git workflow:

1. **Create a Git repository for the Food Delivery Application.**
2. **Add application source code and configuration files.**
3. **Commit changes with meaningful messages.**
4. **Create separate branches for customer, restaurant, payment, and delivery modules.**
5. **Merge tested changes into the main branch.**
6. **Push the repository to GitHub for collaboration and backup.**

Git commands:

```
git init
git add .
git commit -m "Initial food delivery application"
git branch feature-order-management
git checkout feature-order-management
git add .
git commit -m "Add order management module"
git checkout main
git merge feature-order-management
git remote add origin <GitHub-Repository-URL>
git push -u origin main
```

Note: Replace <GitHub-Repository-URL> with the actual URL of the created GitHub repository. Configure the remote only once.

8. DOCKER IMPLEMENTATION

Docker is used to package the application and its dependencies into containers. This helps the application run consistently across development, testing, and production environments.

Docker benefits for this application:

- **Ensures consistent execution across different systems.**
- **Simplifies application setup and deployment.**
- **Separates application services into independent containers.**
- **Makes it easier to scale backend services when demand increases.**
- **Supports automated testing and deployment workflows.**

Sample Dockerfile for a Python Flask backend:

```
FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 5000

CMD ["python", "app.py"]
```

Sample requirements.txt:

```
Flask
```

Docker commands:

```
docker build -t food-delivery-app .
docker run -p 5000:5000 food-delivery-app
```

The Flask application must listen on 0.0.0.0 inside the container to accept connections through the published port. For example, use app.run(host="0.0.0.0", port=5000).

For a complete deployment, the backend, database, and other services can be configured using Docker Compose. Database passwords and payment gateway

credentials should be provided through environment variables or a secrets management system rather than hard-coded in the source code.

9. SECURITY AND RELIABILITY

- **Use HTTPS to protect data transferred between clients and the server.**
- **Apply secure authentication and role-based access control.**
- **Validate customer inputs before processing orders.**
- **Use a trusted payment gateway for payment transactions.**
- **Never store raw card details or sensitive payment credentials in the application database.**
- **Maintain database backups and transaction logs.**
- **Use error handling and monitoring to identify application failures.**
- **Use automated tests before deploying new versions.**
- **Maintain order and payment status consistency when network failures occur.**

10. ADVANTAGES

1. **Scalability: The system can support more customers, restaurants, and delivery partners as demand increases.**
2. **Efficiency: Automated order processing reduces manual work.**
3. **Real-Time Tracking: Customers can monitor order preparation and delivery progress.**
4. **Secure Payments: Payment gateway integration supports verified transactions.**
5. **Maintainability: Modular architecture makes individual components easier to update.**
6. **Version Control: Git tracks changes and supports team collaboration.**
7. **Easy Deployment: Docker provides a consistent environment for testing and production.**
8. **Improved Customer Experience: Notifications and delivery updates keep customers informed.**

11. RESULT

The architecture for the Food Delivery Application is designed successfully by integrating customer management, restaurant management, delivery partner management, order processing, payment processing, notifications, and database services. Git provides source code version control and collaboration, while Docker supports consistent application packaging and deployment.

12. CONCLUSION

The proposed Food Delivery Application architecture provides an organized, secure, and scalable solution for online food ordering and delivery. The modular design enables smooth communication between customers, restaurants, delivery partners, and payment services. By incorporating Git for version control and Docker for containerization, the application becomes easier to develop, test, maintain, and deploy. This architecture can be extended in the future with AI-based food recommendations, optimized delivery routes, and demand prediction.